

**VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**OPERATING SYSTEMS – CO2018
ASSIGNMENT REPORT
SIMPLE OPERATING SYSTEM**

Class: CC02

Instructor: Mr. Hoang Le Hai Thanh

Group: Three-six members (G36)

No.	Full Name	Student ID
1	Huynh Gia Bao	2452121
2	Ta Tuan Khai	2452515
3	Tran Le Tuan Khai	2452517
4	Dang Tuan Kiet	2452620
5	Nguyen Le Quang Truc	2453347

Ho Chi Minh City, June 2026

WORK DISTRIBUTION

Course: Operating Systems – CO2018

Assignment: Simple Operating System

Class: CC02

Instructor: Mr. Hoang Le Hai Thanh

Group: Three-six members (G36)

No.	Name	Student ID	Assigned Tasks	Completion
1	Ta Tuan Khai	2452515	MLQ scheduler, queue system, Gantt chart	100%
2	Nguyen Le Quang Truc	2453347	System call interface and PID-based access	100%
3	Huynh Gia Bao	2452121	Virtual memory, allocation, free-list management	100%
4	Tran Le Tuan Khai	2452517	64-bit paging, page-table mapping, kernel memory	100%
5	Dang Tuan Kiet	2452620	Synchronization, testing, report integration	100%

Note: The table records the intended division of work for the report and implementation review. Technical descriptions in this report are limited to the current simulator behavior and do not claim production-grade kernel features.

Table of Contents

1	Introduction	1
1.1	Assignment Objectives	1
1.2	Scope and Assumptions	1
1.3	Report Organization	2
2	System Architecture Overview	2
2.1	High-Level Architecture	2
2.2	Process Control Block and Execution Model	3
2.3	Kernel Structure	4
2.4	Simulation Flow	4
3	Scheduler: Multi-Level Queue Scheduling	6
3.1	MLQ Design and Priority Model	6
3.2	Queue Management	6
3.3	Scheduler Operations	7
3.4	Actual Gantt Chart from <code>os_2_mlq_paging</code>	7
3.5	Process Priority Distribution	9
3.6	Detailed Execution Intervals	9
4	Kernel Interface and System Calls	11
4.1	System Call Table	11
4.2	System Call Routing	11
4.3	PID-Based Process Lookup	11
4.4	Supported Memory Operations in <code>memmap</code>	12
5	Memory Management and Virtual Memory Layout	13
5.1	Memory Structures	13
5.2	User Memory Allocation	13
5.3	Free Operation	14
5.4	Read and Write Operations	15
5.5	Simplified Page-Fault and Swap Handling	15
5.6	Memory Evidence Printed in <code>os_2_mlq_paging.output</code>	16
5.7	Kernel Memory and User/Kernel Copy	16

6	Multi-Level Paging and 64-Bit Address Translation	17
6.1	64-Bit Paging Layout	17
6.2	Five-Level Page-Table Walk	18
6.3	PTE State Model	18
6.4	Address Translation Example	19
6.5	Design Trade-Off	19
7	Multiprocessor Synchronization	20
7.1	Shared Resources	20
7.2	Queue and Memory Synchronization	20
7.3	Race Condition Example	21
8	Experimental Results and Interpretation	21
8.1	Commands Used	21
8.2	Scheduling Result	22
8.3	Memory Result	22
8.4	System Call Result	23
8.5	Actual Output Excerpt	23
9	Questionnaire Answers	24
10	Conclusion	27
A	Actual Output of os_2_m1q_paging	28
A.1	Process Loading and Initial Allocation	28
A.2	Scheduling and Memory Operation Events	28
A.3	Process Completion and CPU Stop Events	30
	References	30

1 Introduction

1.1 Assignment Objectives

This assignment simulates the core components of a simple operating system. The main objective is to understand how an operating system manages CPU scheduling, process execution, system calls, synchronization, and memory-management operations. The simulator is intentionally simplified so that each major concept can be inspected directly in source code, configuration files, and console output.

The implemented system focuses on the following components:

- Multi-Level Queue (MLQ) process scheduling.
- Queue management for ready and running processes.
- A syscall interface for controlled user-to-kernel interaction.
- Virtual memory management using paging.
- RAM and SWAP simulation through physical memory devices.
- A 64-bit multi-level paging model.
- Basic kernel memory allocation and a simplified cache-pool allocator.
- Synchronization for scheduler and memory-related shared resources.

The implementation is not intended to reproduce a production kernel. Instead, it models the major concepts of process scheduling and memory management in a compact environment. This design follows the educational goal of the operating-systems course, where students connect concepts such as processes, system calls, virtual memory, and paging to concrete implementation mechanisms^[1;2].

1.2 Scope and Assumptions

The report describes behavior observed from the simulator and run notes, avoiding claims beyond the current implementation. The kernel cache is treated as a simplified cache-pool allocator, and paging is described as a simplified FIFO-based replacement/swap path.

Synchronization claims are limited to `queue_lock` for scheduler queues and running lists, `mmvm_lock` for major virtual-memory operations, and timer-side mutexes. No additional memory-device lock is assumed.

1.3 Report Organization

This report is organized as follows. Section 2 summarizes the system architecture and simulation flow. Section 3 explains the MLQ scheduler, process priorities, and the multi-core Gantt chart. Section 4 explains syscall routing and PID-based process lookup. Section 5 presents virtual memory, user memory operations, simplified page fault handling, and kernel memory support. Section 6 explains the 64-bit page-table hierarchy. Section 7 discusses synchronization. Section 8 interprets simulation results. Section 9 answers the assignment questions. Section 10 concludes the report, and Appendix A provides selected simulation output excerpts.

2 System Architecture Overview

2.1 High-Level Architecture

The simulator connects a process loader, CPU worker threads, a scheduler, a syscall layer, and the memory subsystem. Figure 2.1 gives a high-level view of the main modules. The user-process side is represented by process files and PCBs, while kernel-side services include scheduling, syscall dispatch, virtual memory metadata, and physical memory devices.

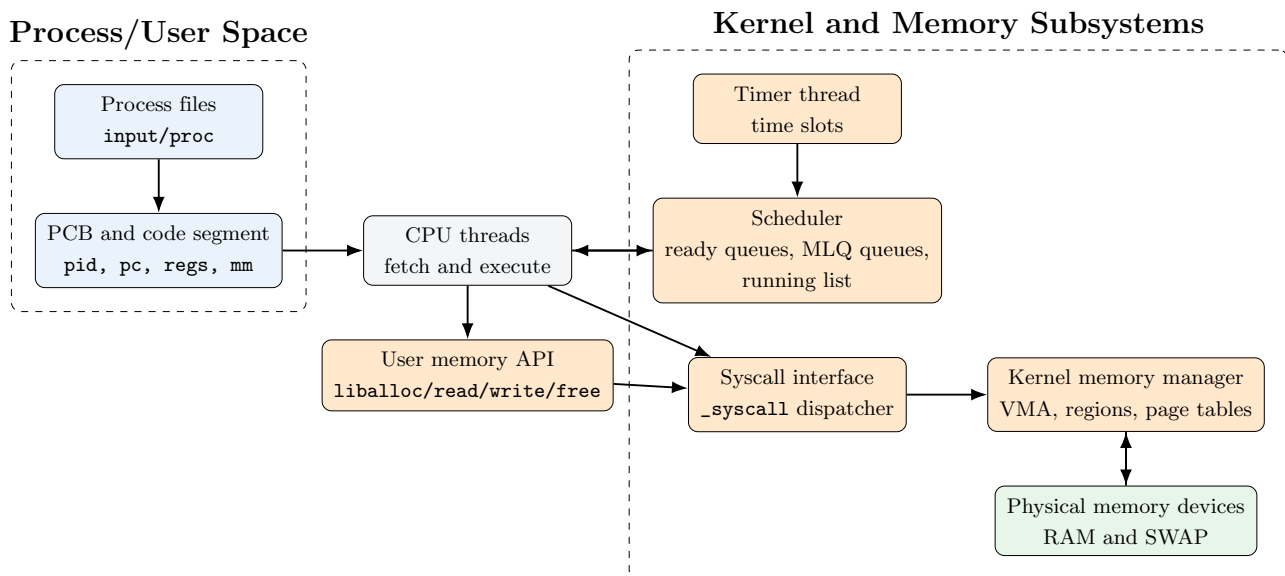


Figure 2.1: General system architecture of the simple OS simulator.

The arrows in Figure 2.1 show that ordinary process execution and privileged memory operations are separated. A process can execute instructions on a CPU thread, but memory mapping and physical I/O are routed through the syscall and memory-management layers.

2.2 Process Control Block and Execution Model

Each process is represented by a Process Control Block (PCB). In the current implementation, the PCB stores process identity, priority, program path, code segment, registers, program counter, kernel reference, and memory-management references. Listing 1 shows the main fields used by the report discussion.

Listing 1: Simplified PCB fields used by the simulator.

```
uint32_t pid;
uint32_t priority;
char path[100];
struct code_seg_t *code;
addr_t regs[10];
uint32_t pc;
#ifdef MLQ_SCHED
uint32_t prio;
#endif
struct krnl_t *krnl;
struct mm_struct *mm;
struct memphy_struct *mram;
struct memphy_struct **mswp;
struct memphy_struct *active_mswp;
```

The process is simulated as a sequence of instructions loaded from a file under `input/proc`. The virtual CPU executes these instructions one by one. Table 2.1 summarizes the main instruction groups.

Table 2.1: Main simulated instruction groups

Instruction	Purpose
<code>calc</code>	Simulate CPU calculation.
<code>alloc</code>	Allocate user-space memory.
<code>free</code>	Free a user-space memory region.
<code>read</code>	Read one byte from user-space memory.
<code>write</code>	Write one byte to user-space memory.
<code>kmalloc</code>	Allocate kernel-space memory.
<code>kmem_cache_create</code>	Create a simplified kernel cache pool.
<code>kmem_cache_alloc</code>	Allocate an object from a cache pool.
<code>copy_from_user</code>	Copy data from user space to kernel space.
<code>copy_to_user</code>	Copy data from kernel space to user space.
<code>syscall</code>	Invoke a kernel service through the syscall interface.

The `pc` field records the instruction index. When `pc` reaches the number of instructions in the code segment, the process is considered finished. If a process has not finished after its time quantum, it is returned to the scheduler so that another process can run.

2.3 Kernel Structure

The kernel structure connects the scheduler and memory-management subsystems. In this project, each PCB points to the kernel through `proc->kernl`. The kernel stores ready queues, MLQ ready queues, the running process list, the kernel memory-management structure, main RAM, SWAP devices, the active SWAP device, and the kernel page-directory reference.

A key design point is that a user process does not directly manipulate kernel memory-management structures through `syscall`. Instead, `syscall` handlers receive a PID and resolve the actual process inside the kernel. This keeps privileged operations under kernel control and avoids exposing raw PCB pointers as a user-level interface.

2.4 Simulation Flow

The simulator starts from an input configuration file, initializes hardware-like components, loads process files, and lets CPU threads repeatedly fetch and execute processes. Figure 2.2 summarizes this execution pipeline.

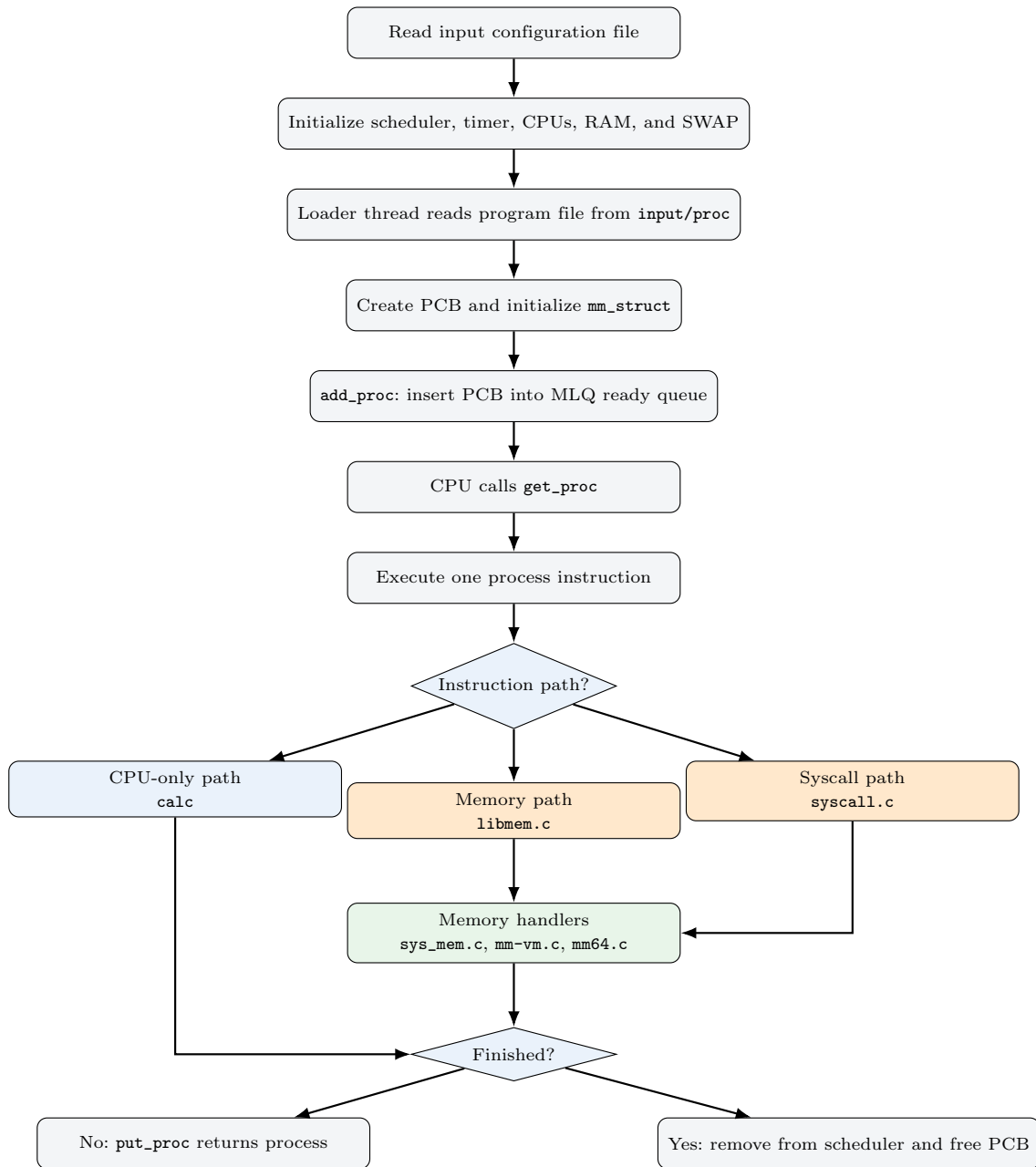


Figure 2.2: Simulation flow from configuration loading to process completion.

Figure 2.2 shows the full simulation cycle: load a process, schedule it, execute one instruction group, perform memory or syscall operations if required, and either requeue or terminate the process.

3 Scheduler: Multi-Level Queue Scheduling

3.1 MLQ Design and Priority Model

The current scheduler uses Multi-Level Queue scheduling. It maintains one queue for each priority level, as shown in Listing 2.

Listing 2: Core MLQ scheduler state.

```
static struct queue_t mlq_ready_queue[MAX_PRIO];
static int slot[MAX_PRIO];
```

The project uses `MAX_PRIO = 140`. Priority values range from 0 to 139. A lower priority number means a higher scheduling priority. Therefore, priority 0 is the highest priority, while priority 139 is the lowest priority.

Each priority level receives a slot budget:

$$\text{slot}[\text{prio}] = \text{MAX_PRIO} - \text{prio}.$$

This means that a process at priority 0 receives the largest CPU share, while a process at priority 139 still receives at least one slot. Table 3.1 shows representative values.

Table 3.1: MLQ priority and slot examples

Priority	Slot count	Meaning
0	140	Highest priority, largest CPU share.
15	125	High priority.
38	102	Medium priority.
120	20	Low priority.
139	1	Lowest priority, but still receives CPU time.

The scheduler implementation uses state variables `curr_prio` and `curr_slot_cnt` to remember which priority level is currently being served and how many slots have already been consumed.

3.2 Queue Management

The basic queue operations are implemented in `queue.c`. Table 3.2 summarizes the queue operations that support the scheduler. In MLQ mode, each queue normally contains processes of the same priority, so the behavior within one priority queue is effectively FIFO.

Table 3.2: Queue operations

Function	Purpose
<code>empty(q)</code>	Check whether a queue is empty.
<code>enqueue(q, proc)</code>	Insert a PCB at the end of the queue.
<code>dequeue(q)</code>	Remove and return process with the highest effective priority.
<code>purgequeue(q, proc)</code>	Remove a specific PCB from a queue.

3.3 Scheduler Operations

The main scheduler functions are shown in Table 3.3. Together, these functions manage process arrival, dispatching, preemption, lookup, and termination.

Table 3.3: Scheduler operations

Function	Description
<code>add_mlq_proc(proc)</code>	Add a newly loaded process into the ready queue matching <code>proc->prio</code> .
<code>get_mlq_proc()</code>	Select a process from the current MLQ queue based on current priority and slot counter.
<code>put_mlq_proc(proc)</code>	Return a preempted process to its MLQ ready queue.

When a CPU dispatches a process, the scheduler places it into `running_list`. When the process finishes, `sched_remove_proc()` removes it before the PCB is freed. This avoids leaving stale process pointers in kernel scheduling structures.

3.4 Actual Gantt Chart from `os_2_mlq_paging`

The following Gantt chart was generated from the provided output file:

Listing 3: Command corresponding to the provided `os_2_mlq_paging.output` file.

```
./os os_2_mlq_paging > os_2_mlq_paging.output
```

The output does not print the input configuration directly, so this section uses only events observed in `os_2_mlq_paging.output`. Four CPU threads are observed in the trace, and six processes are loaded. Table 3.4 lists the process load events printed by the output.

Table 3.4: Observed process load events in `os_2_mlq_paging`

PID	Program	Loaded time slot	Priority
1	m2s	5	120
2	s0	8	38
3	s3	10	39
4	m2s	13	15
5	s2	15	120
6	s1	17	0

Lower numeric priority means higher scheduling priority. Therefore, PID 6 has the highest scheduling priority, while PID 1 and PID 5 are lower-priority processes.

Figure 3.1 visualizes the compressed execution intervals parsed from the output. Idle CPU periods are shaded in gray, and periods after a CPU stop event are marked separately. The detailed repeated slices are preserved in Table 3.7.

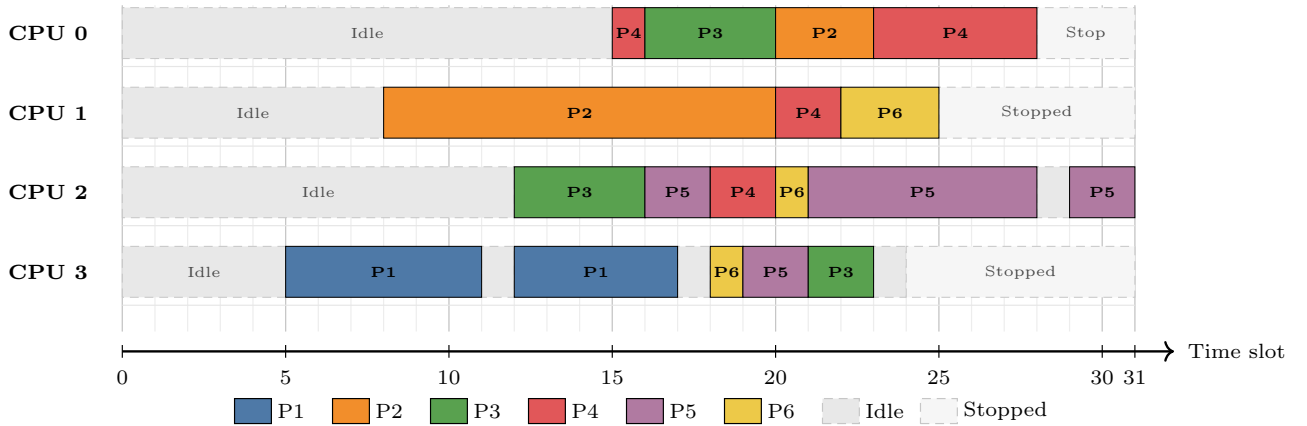


Figure 3.1: Gantt Chart of Multi-Core MLQ Scheduling for `os_2_mlq_paging`, including idle and stopped CPU periods.

The execution shows that all six loaded processes receive CPU time. PID 6 has the highest priority value in scheduling terms because its numeric priority is 0, while PID 1 and PID 5 are lower-priority processes with priority 120. The output also shows that scheduling order is affected by concurrent loader and CPU threads, so exact dispatch times may vary across runs.

Some processes finish immediately after dispatch at the same time slot; these zero-duration completion events are omitted from the visual blocks but preserved in Table 3.8.

3.5 Process Priority Distribution

The process priorities used by the `os_2_mlq_paging` scenario are listed in Table 3.5. The priority grouping rule used here is: 0 = Highest, 1–20 = High, 21–60 = Medium, 61–119 = Low, and 120–139 = Very low.

Table 3.5: Process priority distribution in `os_2_mlq_paging`

PID	Program	Loaded time slot	Priority	Priority group
P1	m2s	5	120	Very low
P2	s0	8	38	Medium
P3	s3	10	39	Medium
P4	m2s	13	15	High
P5	s2	15	120	Very low
P6	s1	17	0	Highest

Figure 3.2 visualizes the same data as a horizontal bar chart. A shorter bar means a higher scheduling priority because lower numeric values represent higher priority.

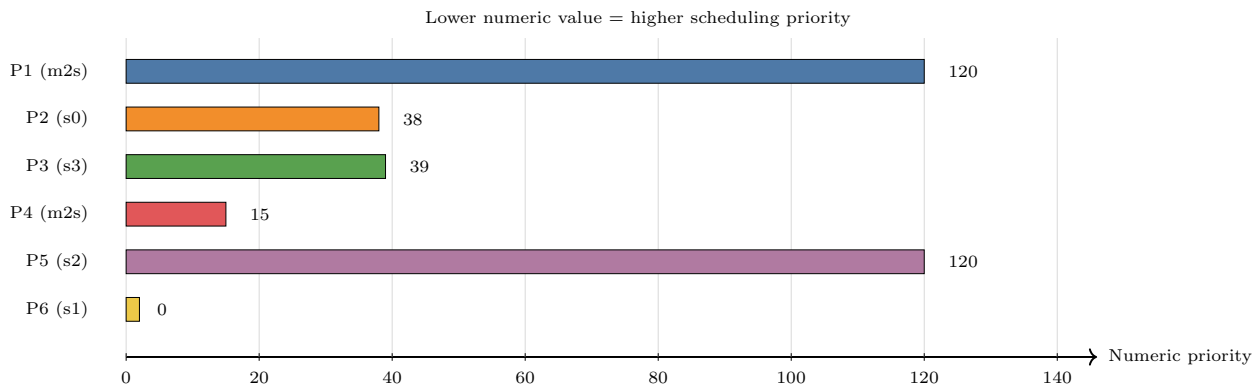


Figure 3.2: Priority Distribution of Processes in `os_2_mlq_paging`.

3.6 Detailed Execution Intervals

Table 3.6 lists the compressed process-execution intervals used for the colored process bars in Figure 3.1. Adjacent repeated slices of the same process on the same CPU are merged in this table for readability.

Table 3.6: Compressed execution intervals used for the Gantt chart

CPU	Execution intervals
CPU 0	15–16: P4 → 16–20: P3 → 20–23: P2 → 23–28: P4
CPU 1	8–20: P2 → 20–22: P4 → 22–25: P6
CPU 2	12–16: P3 → 16–18: P5 → 18–20: P4 → 20–21: P6 → 21–28: P5 → 29–31: P5
CPU 3	5–11: P1 → 12–17: P1 → 18–19: P6 → 19–21: P5 → 21–23: P3

Table 3.7 gives the detailed intervals, including repeated time slices that are merged in Table 3.6.

Table 3.7: Detailed execution intervals parsed from `os_2_mfq_paging.output`

CPU	Start	End	Process	CPU	Start	End	Process
3	5	7	P1	3	18	19	P6
3	7	9	P1	1	19	20	P2
1	8	11	P2	0	19	20	P3
3	9	11	P1	3	19	21	P5
1	11	13	P2	2	18	20	P4
2	12	13	P3	2	20	21	P6
3	12	13	P1	1	20	22	P4
1	13	15	P2	0	20	23	P2
2	13	16	P3	3	21	23	P3
3	13	15	P1	2	21	24	P5
0	15	16	P4	1	22	24	P6
1	15	16	P2	0	23	24	P4
3	15	17	P1	2	24	26	P5
2	16	18	P5	1	24	25	P6
1	16	19	P2	0	24	26	P4
0	16	19	P3	2	26	28	P5
				0	26	28	P4
				2	29	31	P5

Table 3.8: Zero-duration completion events printed at time slot 24

Time slot	CPU	Process	Output event
24	CPU 2	P3	Processed 3 has finished
24	CPU 3	P2	Processed 2 has finished

4 Kernel Interface and System Calls

4.1 System Call Table

The current syscall table contains two main entries, shown in Table 4.1. The syscall interface is used to enter kernel-level operations from process execution. The main memory-related syscall is `mmap`, identified by syscall number 17.

Table 4.1: Current syscall table

Number	Name	Handler
0	<code>listsyscall</code>	<code>sys_listsyscall</code>
17	<code>mmap</code>	<code>sys_mmap</code>

4.2 System Call Routing

The syscall path begins at a process instruction and ends at a kernel handler. Figure 4.1 shows this routing path. The current memory syscall implementation resolves the caller through `sched_find_proc_by_pid(krnl, pid)` instead of trusting a direct PCB pointer supplied by a caller.

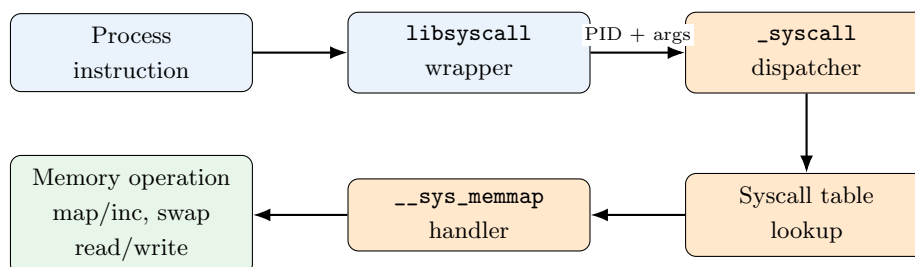


Figure 4.1: System Call Routing Path.

The design in Figure 4.1 keeps user-level requests separated from the internal PCB representation. The syscall layer receives a PID and arguments, then the kernel side checks whether that PID corresponds to a valid process currently managed by the scheduler.

4.3 PID-Based Process Lookup

Inside `sys_mem.c`, the implementation resolves the calling process by PID:

Listing 4: PID-based caller lookup used by the memory syscall handler.

```
caller = sched_find_proc_by_pid(krnl, pid);
```

This design ensures that the kernel resolves the actual process using scheduler-managed process lists. If the PID cannot be resolved, the syscall fails. Table 4.2 compares this design with a direct PCB pointer design.

Table 4.2: PID-based syscall design vs direct PCB pointer

Criterion	PID-based lookup	Direct PCB pointer
Security	Caller provides an identifier; kernel decides whether it maps to a valid process.	Caller passes an internal object address, which is unsafe as a syscall-level interface.
Validation	<code>sched_find_proc_by_pid(krnl, pid)</code> can reject unknown PIDs.	Requires trusting or separately validating a pointer value.
Corruption risk	Lower risk of fake PCB objects because the scheduler lists are the source of truth.	Higher risk if a stale or invalid pointer is used.
Performance	Requires searching scheduler-managed process lists.	Avoids lookup cost if the pointer is valid.
Suitability for this assignment	Better fit because it demonstrates a controlled kernel boundary.	Less suitable because it exposes kernel internals directly.

4.4 Supported Memory Operations in `mmap`

The current `mmap` syscall handles the operations in Table 4.3. This unified kernel interface keeps user-space memory functions simple while still forcing privileged memory operations to pass through the kernel.

Table 4.3: Supported `mmap` operations

Operation	Meaning
<code>SYSTEM_MAP_OP</code>	Map a memory range.
<code>SYSTEM_INC_OP</code>	Increase a VMA limit.
<code>SYSTEM_SWP_OP</code>	Swap a page between RAM and SWAP.
<code>SYSTEM_IO_READ</code>	Read a byte from physical memory.
<code>SYSTEM_IO_WRITE</code>	Write a byte to physical memory.

5 Memory Management and Virtual Memory Layout

5.1 Memory Structures

The virtual memory subsystem is organized through the hierarchy shown in Figure 5.1. Each process has its own `mm_struct`, which provides process-level memory isolation in the simulator.

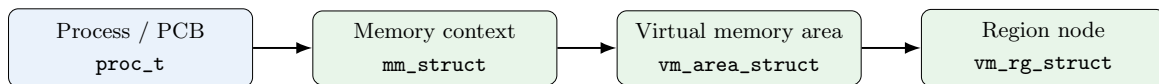


Figure 5.1: Virtual memory structure used by each simulated process.

The main memory structures are summarized in Table 5.1. Figure 5.1 and Table 5.1 together show how a process-level memory context is divided into virtual areas and allocated/free regions.

Table 5.1: Memory-management structures

Structure	Role
<code>mm_struct</code>	Represents a process or kernel memory-management context.
<code>vm_area_struct</code>	Represents a continuous virtual memory area.
<code>vm_rg_struct</code>	Represents an allocated or free region inside a VMA.
<code>mempy_struct</code>	Represents a physical memory device such as RAM or SWAP.
<code>framephy_struct</code>	Represents one physical frame number.

5.2 User Memory Allocation

User-space allocation starts from `liballoc()` and resolves to `__alloc()`. Figure 5.2 shows the allocation logic. The allocator first tries to reuse a free region; only if no suitable region exists does it request a VMA extension through the memory syscall path.

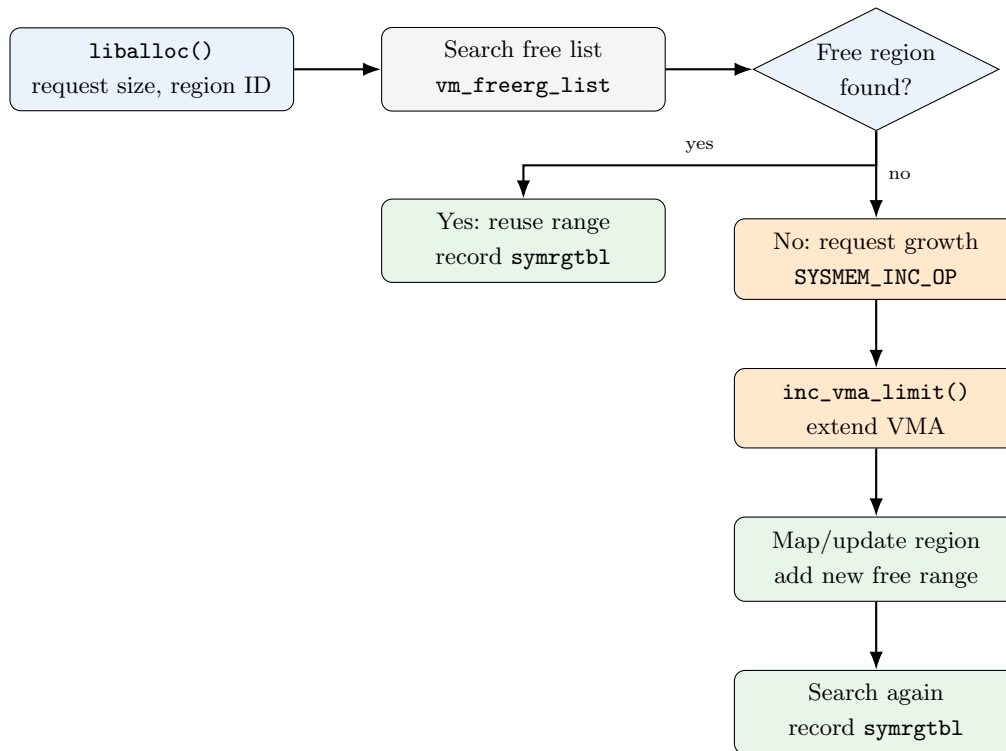


Figure 5.2: ALLOC flow for user-space memory allocation.

In Figure 5.2, reusing a free region is preferred because it avoids growing the VMA unnecessarily. If no free region is available, `SYSMEM_INC_OP` leads to `inc_vma_limit()` in `mm-vm.c`, which extends the virtual memory area and makes the new range available for allocation.

5.3 Free Operation

`libfree()` calls `__free()`. The free operation does not immediately destroy the page-table mapping. Instead, it marks the region as free and adds it back to `vm_freerg_list`. The simplified free flow is shown in Listing 5.

Listing 5: Simplified free-region flow.

```

libfree(proc, reg_index)
-> __free(proc, vmaid, rgid)
-> locate symrgtbl[rgid]
-> create a free-region node
-> add node to vm_freerg_list
-> clear symrgtbl[rgid]

```

This approach allows later allocation requests to reuse previously released virtual regions.

5.4 Read and Write Operations

`libread()` and `libwrite()` validate the requested region and offset before accessing physical memory. Figure 5.3 shows that both operations share the same page-resolution stage through `pg_getpage()`.

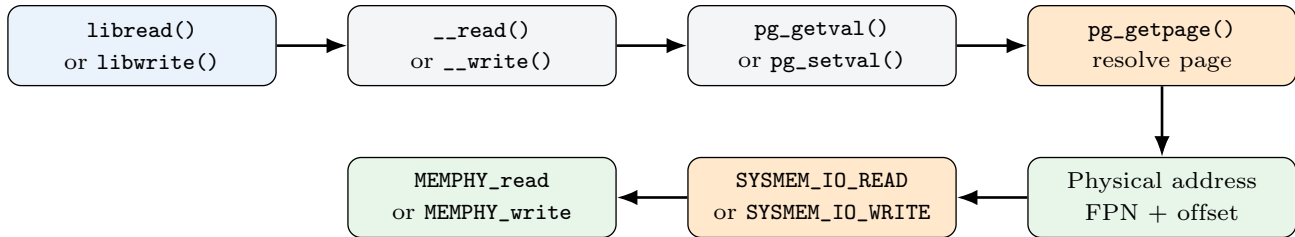


Figure 5.3: READ/WRITE flow from library call to physical memory access.

The physical address is computed as:

$$\text{Physical Address} = (\text{FPN} \ll \text{PAGING_ADDR_FPN_LOBIT}) + \text{offset}.$$

If the requested page is not present, `pg_getpage()` attempts to resolve the page before the read or write continues.

5.5 Simplified Page-Fault and Swap Handling

The implementation provides a simplified page-fault and swap path suitable for demonstrating FIFO-based replacement in the simulator. It should not be read as a full production demand-paging implementation. Figure 5.4 shows the conceptual path used in the report.

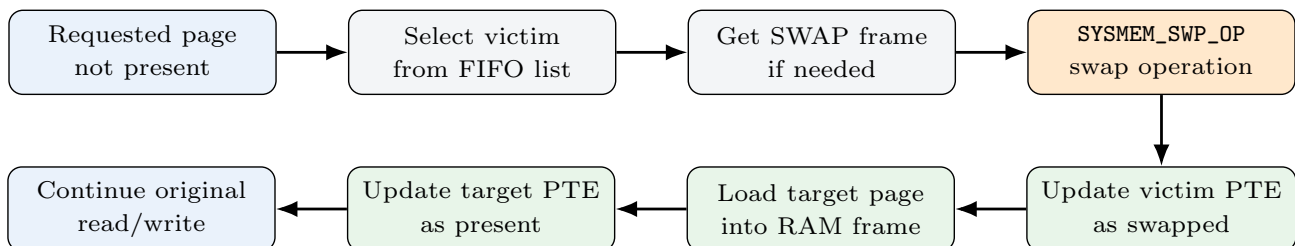


Figure 5.4: Simplified FIFO-based page-fault and swap handling path.

The replacement policy described in Figure 5.4 is FIFO. The FIFO list is maintained through `fifo_pgn` in `mm_struct`. This is enough for the assignment to demonstrate the relationship between page presence, swap state, and physical frame reuse.

5.6 Memory Evidence Printed in `os_2_mlq_paging.output`

The provided `os_2_mlq_paging.output` file prints user allocation and free events for the two `m2s` processes. It does not print detailed VMA fields such as `vm_start`, `sbrk`, or internal page-table entries. Therefore, Table 5.2 records only memory events that are directly visible in the output.

Table 5.2: Memory events directly printed in `os_2_mlq_paging.output`

Operation	Evidence from output	Interpretation
<code>alloc</code>	<code>[LIBALLOC] PID 1: alloc [300] [0]</code>	PID 1 allocates 300 bytes into region/register 0.
<code>alloc</code>	<code>[LIBALLOC] PID 1: alloc [100] [1]</code>	PID 1 allocates another 100-byte user region.
<code>free</code>	<code>[LIBFREE] PID 1: free [0]</code>	PID 1 releases region/register 0.
<code>alloc</code>	<code>[LIBALLOC] PID 4: alloc [300] [0]</code>	PID 4 runs the same <code>m2s</code> workload and allocates memory.
<code>alloc</code>	<code>[LIBALLOC] PID 4: alloc [100] [1]</code>	PID 4 creates a second user region.
<code>free</code>	<code>[LIBFREE] PID 4: free [0]</code>	PID 4 frees the first region.
<code>free</code>	<code>[LIBFREE] PID 4: free [1]</code>	PID 4 frees the second region.

Table 5.2 supports the allocation and free flow in Figure 5.2. Detailed VMA addresses and page-table state are not printed in the provided output, so they are not used as direct experimental evidence in this report.

5.7 Kernel Memory and User/Kernel Copy

The project supports a simple kernel memory path through `kmalloc`, `kmem_cache_create`, `kmem_cache_alloc`, `copy_from_user`, and `copy_to_user`. The cache mechanism is described here as a simplified kernel cache-pool allocator. It does not attempt to fully reproduce a production slab allocator; instead, it pre-allocates a pool and returns available object slots in a simple manner suitable for the assignment.

The copy functions avoid direct unsafe access across user/kernel boundaries. The flow of `copy_from_user` reads bytes from a user region and writes them into a kernel region. The flow of `copy_to_user` reads bytes from a kernel region and writes them back to a user region. Listing 6 shows the `m2s` workload that exercises this path.

Listing 6: Kernel memory and user/kernel copy workload in `m2s`.

```
alloc 300 0
alloc 100 1
free 0
kmalloc 200 2
kmem_cache_create 300 10 1
kmem_cache_alloc 3 1
```

```
copy_from_user 1 4 0 3
kmalloc 200 4
copy_to_user 3 4 0 3
free 2
free 1
free 3
```

The input `os_2_m1q_paging` uses `m2s`, confirming that the kernel memory instructions can be loaded and executed in the simulator configuration used by the report.

6 Multi-Level Paging and 64-Bit Address Translation

6.1 64-Bit Paging Layout

The simulator uses a 64-bit address model with five paging levels. Figure 6.1 shows the bit layout used in the report. The page offset is 12 bits, so the page size is $2^{12} = 4096$ bytes.

63–57 Space tag	56–48 PGD	47–39 P4D	38–30 PUD	29–21 PMD	20–12 PT	11–0 Offset
--------------------	--------------	--------------	--------------	--------------	-------------	----------------

64-bit virtual address fields

Figure 6.1: 64-bit virtual address layout used by the paging simulator.

The same layout is summarized in Table 6.1. Figure 6.1 is easier to read visually, while the table gives exact field names and bit ranges.

Table 6.1: 64-bit virtual-address layout

Field	Bits
Space tag / high bits	63–57
PGD	56–48
P4D	47–39
PUD	38–30
PMD	29–21
PT	20–12
Offset	11–0

6.2 Five-Level Page-Table Walk

The function `mm64_page_walk()` performs a five-level walk. Figure 6.2 shows the conceptual traversal from the top-level directory to the final page-table entry. If `create` is enabled and an intermediate table is missing, the function dynamically allocates a new 512-entry table using `calloc`.

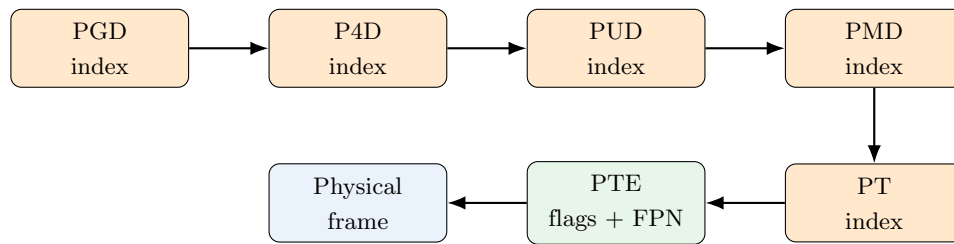


Figure 6.2: 5-level page table walk from PGD to final frame information.

Figure 6.2 illustrates sparse page-table allocation. Instead of allocating a huge flat table for the entire address space, the simulator only allocates intermediate page-table levels when a virtual page is actually mapped.

6.3 PTE State Model

A page-table entry records the status of a virtual page. Figure 6.3 shows the practical state transitions used in the report discussion. A page may be unmapped, present in RAM, swapped out, or brought back into RAM.

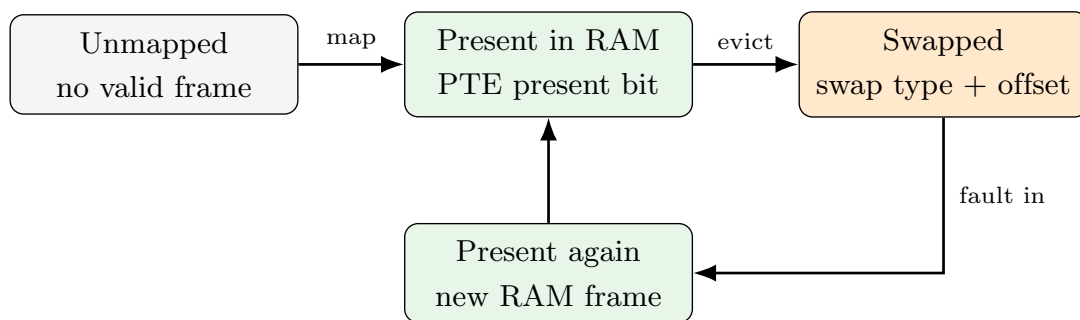


Figure 6.3: PTE state diagram for present and swapped pages.

The important fields and flags behind Figure 6.3 are summarized in Table 6.2.

Table 6.2: Important PTE fields and flags

Flag / Field	Meaning
Present	Page currently has a RAM frame.
Swapped	Page has been moved to SWAP.
Dirty	Page has been modified.
FPN	Physical frame number.
Swap type / offset	Location in a SWAP device.

6.4 Address Translation Example

Consider the following virtual address: $VA = 0x0000123456789ABC$.

The fields are extracted by the bit operations in Listing 7.

Listing 7: Index and offset extraction pattern.

```
index = (VA >> shift) & 0x1FF;
offset = VA & 0xFFF;
```

The decomposition is shown in Table 6.3. The page-table walk line below the table follows the same structure as Figure 6.2.

Table 6.3: Example virtual-address decomposition

Field	PGD	P4D	PUD	PMD	PT	Offset
Value	0x000	0x024	0x0D1	0x0B3	0x189	0xABC

Assume the final PTE contains **Present** = 1 and **FPN** = 0x1234. Then the physical address is:

$$\begin{aligned}
 PA &= (0x1234 \ll 12) + 0xABC \\
 &= 0x1234000 + 0xABC \\
 &= 0x1234ABC.
 \end{aligned}$$

6.5 Design Trade-Off

The implemented five-level paging design has both benefits and costs, summarized in Table 6.4. For this simulator, five-level paging is suitable because it demonstrates modern address translation while avoiding the memory cost of a huge flat page table.

Table 6.4: Five-level paging trade-off

Aspect	Advantage	Cost
Storage	Allocates only used page-table paths.	More pointer levels.
Translation	Deterministic fixed-depth walk.	Up to five directory accesses.
Scalability	Suitable for large sparse virtual spaces.	More complex than a flat table.
Swap support	PTE encode present/swapped state.	More metadata management.

7 Multiprocessor Synchronization

7.1 Shared Resources

Since the simulator can run multiple CPU threads, several structures may be accessed concurrently. These shared resources must be protected to avoid race conditions. Table 7.1 summarizes the major shared resources and their protection mechanisms.

Table 7.1: Shared Resources and Locks

Shared resource	Location	Protection
MLQ ready queues	<code>sched.c</code>	<code>queue_lock</code>
Running process list	<code>sched.c</code>	<code>queue_lock</code>
Slot state / scheduler state	<code>sched.c</code>	<code>queue_lock</code>
Virtual memory metadata	<code>libmem.c</code>	<code>mmvm_lock</code>
Timer events	<code>timer.c</code>	Timer mutexes

7.2 Queue and Memory Synchronization

The scheduler uses `queue_lock` to protect adding a process to a ready queue, removing a process from a ready queue, returning a preempted process, searching process lists by PID, and removing a finished process from lists. Without this lock, two CPU threads could dispatch the same process at the same time or corrupt a queue while shifting entries.

The memory library uses `mmvm_lock` to protect major user memory operations: allocation, freeing, reading, writing, and updates to virtual memory regions and symbol-table entries. Without this lock, two CPUs could allocate overlapping regions or corrupt `vm_freerg_list`.

7.3 Race Condition Example

Figure 7.1 shows a simple example in which two CPUs attempt allocation at the same time. The upper panel shows the race that may occur without a lock, while the lower panel shows how `mmvm_lock` serializes the update.

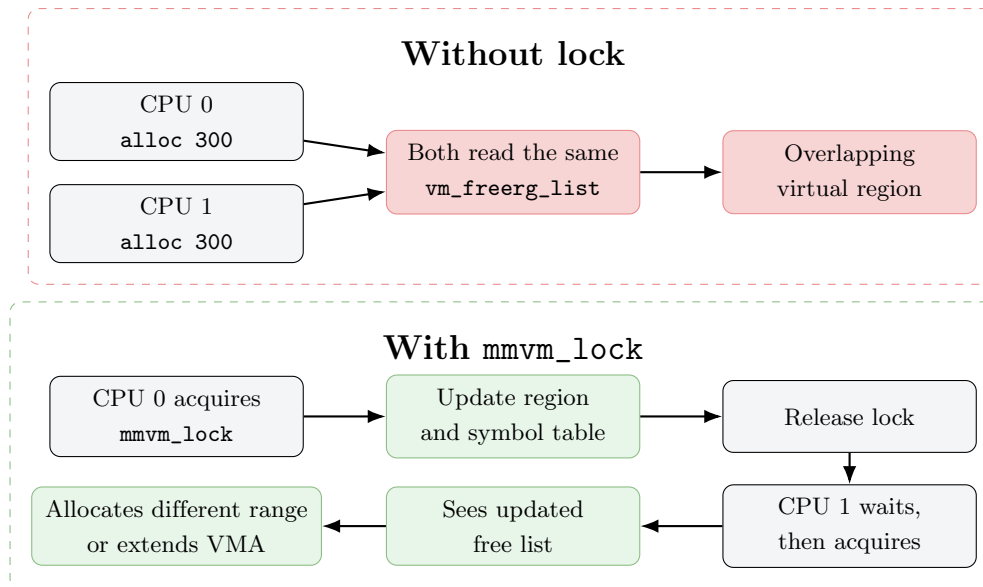


Figure 7.1: Race condition example without locks and serialized allocation with `mmvm_lock`.

Figure 7.1 illustrates why coarse-grained locking is acceptable for this simulator: clarity and correctness are more important than fine-grained parallel memory performance in the assignment context.

8 Experimental Results and Interpretation

8.1 Commands Used

The main experimental scenario is `os_2_mlq_paging`. This scenario is used because it includes multiple CPU threads, six loaded processes, MLQ priorities ranging from 0 to 120, user-space allocation/free operations, and the `m2s` workload that contains kernel-memory-related instructions. The report uses the provided output file `os_2_mlq_paging.output` as its direct experimental source.

Listing 8: Commands relevant to the updated experiment.

```
make clean && make all
./os os_2_mlq_paging
./os os_syscall_list
```

Table 8.1 summarizes the purpose and observed result of each command. The main run is not reinterpreted using older `os_1` data.

Table 8.1: Test Summary Table

Command	Purpose	Observed result
<code>make clean && make all</code>	Compile simulator.	Build completed successfully.
<code>./os os_2_mlq_paging</code>	Main scheduling and memory experiment.	Six processes loaded and CPUs stopped after completion.
<code>./os os_syscall_list</code>	Verify syscall table dispatch.	Syscall list can be invoked.

8.2 Scheduling Result

The output confirms that all six loaded processes eventually finish. Table 8.2 lists the completion events printed in `os_2_mlq_paging.output`.

Table 8.2: Scheduling Result for `os_2_mlq_paging`

PID	Program	Priority	Observed completion
1	m2s	120	Finished at time slot 17 on CPU 3.
2	s0	38	Finished at time slot 24 on CPU 3.
3	s3	39	Finished at time slot 24 on CPU 2.
4	m2s	15	Finished at time slot 28 on CPU 0.
5	s2	120	Finished at time slot 31 on CPU 2.
6	s1	0	Finished at time slot 25 on CPU 1.

Lower-priority processes such as PID 1 and PID 5 still receive CPU time, while higher-priority processes such as PID 6 are also dispatched after being loaded. This behavior is consistent with the slot-based MLQ design described in Section 3.

8.3 Memory Result

The memory logs printed in `os_2_mlq_paging.output` are allocation and free events for PID 1 and PID 4, both of which execute the `m2s` program. Table 8.3 lists only direct evidence from the provided output.

Table 8.3: Memory operation evidence from `os_2_mlq_paging.output`

Operation	Evidence from output	Interpretation
alloc	[LIBALLOC] PID 1: alloc [300] [0]	PID 1 allocates 300 bytes into region/register 0.
alloc	[LIBALLOC] PID 1: alloc [100] [1]	PID 1 allocates another 100-byte region.
free	[LIBFREE] PID 1: free [0]	PID 1 releases region/register 0.
alloc	[LIBALLOC] PID 4: alloc [300] [0]	PID 4 runs the same <code>m2s</code> workload and allocates memory.
alloc	[LIBALLOC] PID 4: alloc [100] [1]	PID 4 creates a second user region.
free	[LIBFREE] PID 4: free [0]	PID 4 frees the first region.
free	[LIBFREE] PID 4: free [1]	PID 4 frees the second region.

The provided output mainly prints user allocation and free logs. The `m2s` program also contains kernel-memory instructions, but if their debug output is not printed, the report should not claim detailed kernel-memory trace lines from this output. Instead, the report describes them as part of the executed workload design. Since PID 1 and PID 4 both execute the `m2s` program, this scenario is suitable for testing the code path that includes user allocation/free and kernel-memory-related instructions. However, only operations that appear in the output are used as direct experimental evidence.

8.4 System Call Result

The syscall subsystem is validated separately using `os_syscall_list`. In the main `os_2_mlq_paging` run, memory-related operations still go through the `memmap` syscall internally when VMA extension or physical memory I/O is required. The main output file does not print a detailed syscall trace for every memory operation, so this report does not invent such lines.

8.5 Actual Output Excerpt

Listing 9 shows a short excerpt from `os_2_mlq_paging.output`. The full selected output is included in Appendix A.

Listing 9: Actual Output Excerpt from `os_2_mlq_paging`.

```
Time slot 5
    Loaded a process at input/proc/m2s, PID: 1 PRI0: 120
    CPU 3: Dispatched process 1
Time slot 6
[LIBALLOC] PID 1: alloc [300] [0]
[LIBALLOC] PID 1: alloc [100] [1]
Time slot 7
    CPU 3: Put process 1 to run queue
    CPU 3: Dispatched process 1
libfree:219
[LIBFREE] PID 1: free [0]
```

```

Time slot 8
    Loaded a process at input/proc/s0, PID: 2 PRI0: 38
    CPU 1: Dispatched process 2
...
Time slot 17
    Loaded a process at input/proc/s1, PID: 6 PRI0: 0
    CPU 3: Processed 1 has finished
...
Time slot 24
    CPU 2: Processed 3 has finished
    CPU 3: Processed 2 has finished
    CPU 3 stopped
Time slot 25
    CPU 1: Processed 6 has finished
    CPU 1 stopped
Time slot 28
    CPU 0: Processed 4 has finished
    CPU 0 stopped
Time slot 31
    CPU 2: Processed 5 has finished
    CPU 2 stopped

```

9 Questionnaire Answers

Question 1: What are the advantages and disadvantages of using a unified system call interface?

A unified system call interface means that different resources can be accessed through a consistent set of kernel entry points. For example, memory, file, and I/O operations can share a similar read/write style interface.

Table 9.1 summarizes the main advantages and disadvantages.

Table 9.1: Advantages and disadvantages of a unified system call interface

Advantages	Disadvantages
Simplifies application design because user programs do not need hardware-specific details.	Adds overhead because requests pass through generic syscall dispatching before reaching a specific handler.
Improves portability because the same interface can be used across different resource types.	Some resources do not naturally fit a read/write/free model.
Makes the kernel more modular because each subsystem can implement a common operation interface.	Makes error handling more complex because different subsystems may fail for different reasons.
Supports controlled access because privileged operations must pass through the kernel.	May reduce efficiency compared with specialized low-level access.

In this simulator, the unified memory syscall **memmap** simplifies the design by routing map, increase, swap, read, and write operations through a single kernel interface.

Question 2: When a system call executes for too long, how does the OS detect and handle it?

A real operating system detects long-running system calls mainly through timer interrupts and watchdog mechanisms. Timer interrupts periodically return control to the kernel scheduler. If a system call blocks while waiting for I/O, the process can be placed into a blocked state so that other processes can run.

Common handling methods include kernel preemption if the kernel supports it, blocking the process until the resource becomes available, returning timeout errors, interrupting the syscall through signals, and killing the process or triggering a kernel panic in severe kernel lockup cases. In this simulator, long execution is simplified through time slicing. CPU threads execute a process for a fixed time quantum and then return it to the scheduler if it has not finished.

Question 3: What is the benefit of each MLQ detailed policy?

The MLQ scheduler uses separate queues for each priority level. This helps the OS organize processes according to importance. The policy has the following benefits:

1. Multiple ready queues separate high-priority and low-priority workloads.
2. The slot formula $\text{slot} = \text{MAX_PRIO} - \text{prio}$ gives high-priority processes more CPU share.
3. Low-priority queues still receive at least some CPU time, reducing starvation.
4. Round-robin behavior within the same priority level gives fairness among equal-priority processes.
5. The live priority field allows the execution configuration to override the default priority from the program file.

Question 4: Why combine segmentation with paging?

Segmentation provides a logical view of memory, while paging provides efficient physical memory management. Segmentation helps represent meaningful program areas such as code, heap, stack, and data. However, pure segmentation requires contiguous allocation and can suffer from external fragmentation.

Paging divides memory into fixed-size pages and frames. This removes external fragmentation and allows non-contiguous allocation. However, paging alone does not naturally represent logical program components. Combining both ideas gives the system the benefits of logical organization and flexible physical mapping: VMA structures organize virtual memory logically, paging maps virtual pages to physical frames efficiently, and the OS can grow memory regions while still mapping them to non-contiguous physical frames.

Question 5: What are the benefits of extending hierarchical paging to N levels?

Extending paging to multiple levels has several benefits:

- It avoids allocating one enormous flat page table.
- It supports sparse allocation because page-table levels are created only when needed.
- It fits naturally with page-sized hardware units.
- It supports large virtual address spaces.
- It allows deterministic translation through a fixed hierarchy.

The cost is that address translation requires multiple memory accesses during a page-table walk. In real systems, this cost is reduced by a Translation Lookaside Buffer (TLB). In this simulator, multi-level paging is useful because it demonstrates how large virtual address spaces can be represented without allocating a full flat page table.

Question 6: What are the advantages and disadvantages of paging and contiguous allocation?

Contiguous allocation places each process in one continuous physical memory region. Its advantages are simple implementation, fast address calculation, easy protection using base and limit, and low metadata overhead. Its disadvantages are external fragmentation, difficult process growth, possible need for compaction, and lack of virtual-memory support.

Paging divides virtual memory into pages and physical memory into frames. Its advantages are no external fragmentation, virtual-memory support, flexible process growth, non-contiguous physical allocation, and page-level protection and sharing. Its disadvantages are internal fragmentation inside the last page, page-table memory overhead, slower translation if no TLB is available, and more complex page-fault and replacement logic.

Question 7: What happens if synchronization is not handled?

If synchronization is missing, multiple CPU threads may access shared kernel structures at the same time. This can cause race conditions. For example, two CPUs may dequeue the same process, one CPU may shift the queue array while another CPU is inserting into it, two memory operations may modify the same free-region list at the same time, a process may be freed while its pointer is still in the running list, and page or memory metadata may become inconsistent.

The current implementation uses `queue_lock` for scheduler structures and `mmvm_lock` for major virtual memory operations. These locks help ensure that queue operations and memory metadata updates occur atomically.

10 Conclusion

The current implementation completes the main assignment goals at the simulator level. It supports MLQ scheduling, process queue management, syscall-based kernel entry, PID-based process lookup, user memory allocation, memory free and access operations, 64-bit multi-level paging, and basic kernel memory/cache operations.

The experimental results show that all six loaded processes in the `os_2_mlq_paging` scenario are loaded, scheduled, and completed. The Gantt chart in Figure 3.1 confirms that the scheduler distributes execution across the four observed CPU threads while respecting process priorities. The printed memory evidence demonstrates user allocation and free events, while lower-level VMA and page-table details are not printed in the provided output.

Overall, this project helps illustrate the core OS concepts of scheduling, virtual memory, system calls, and synchronization in a simplified but executable operating-system simulator. Future improvements could include richer syscall coverage, more detailed fault handling, additional page replacement algorithms, stronger validation tests, and improved tracing tools for debugging scheduler and memory behavior.

A Actual Output of os_2_mlq_paging

This appendix contains the output evidence used for the updated experimental sections. The output is split into three parts for readability: process loading and initial allocation, scheduling and memory-operation events, and completion/CPU-stop events.

A.1 Process Loading and Initial Allocation

Listing 10 shows the beginning of `os_2_mlq_paging.output`, including the first `m2s` load event and the printed allocation/free logs for PID 1.

Listing 10: Process loading and initial allocation events from `os_2_mlq_paging.output`.

```
Time slot 0
ld_routine
Time slot 1
Time slot 2
Time slot 3
Time slot 4
Time slot 5
    Loaded a process at input/proc/m2s, PID: 1 PRI0: 120
    CPU 3: Dispatched process 1
Time slot 6
[LIBALLOC] PID 1: alloc [300] [0]
[LIBALLOC] PID 1: alloc [100] [1]
Time slot 7
    CPU 3: Put process 1 to run queue
    CPU 3: Dispatched process 1
libfree:219
[LIBFREE] PID 1: free [0]
Time slot 8
    Loaded a process at input/proc/s0, PID: 2 PRI0: 38
    CPU 1: Dispatched process 2
Time slot 9
    CPU 3: Put process 1 to run queue
    CPU 3: Dispatched process 1
Time slot 10
    Loaded a process at input/proc/s3, PID: 3 PRI0: 39
Time slot 11
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 2
    CPU 3: Put process 1 to run queue
Time slot 12
    CPU 3: Dispatched process 1
    CPU 2: Dispatched process 3
Time slot 13
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 2
    CPU 3: Put process 1 to run queue
    CPU 2: Put process 3 to run queue
    CPU 2: Dispatched process 3
    CPU 3: Dispatched process 1
    Loaded a process at input/proc/m2s, PID: 4 PRI0: 15
```

A.2 Scheduling and Memory Operation Events

Listing 11 shows the middle part of the output. It includes the second `m2s` process, PID 4, and the printed allocation/free evidence used in Table 8.3.

Listing 11: Scheduling and memory-operation events from `os_2_mfq_paging.output`.

```
Time slot 14
Time slot 15
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 2
    CPU 0: Dispatched process 4
[LIBALLOC] PID 4: alloc [300] [0]
    CPU 3: Put process 1 to run queue
    Loaded a process at input/proc/s2, PID: 5 PRI0: 120
    CPU 3: Dispatched process 1
libfree:219
[LIBFREE] PID 1: free [1]
Time slot 16
    CPU 2: Put process 3 to run queue
    CPU 2: Dispatched process 5
[LIBALLOC] PID 4: alloc [100] [1]
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 2
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 3
Time slot 17
    Loaded a process at input/proc/s1, PID: 6 PRI0: 0
    CPU 3: Processed 1 has finished
Time slot 18
    CPU 3: Dispatched process 6
    CPU 2: Put process 5 to run queue
    CPU 2: Dispatched process 4
libfree:219
[LIBFREE] PID 4: free [0]
Time slot 19
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 2
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 3
    CPU 3: Put process 6 to run queue
    CPU 3: Dispatched process 5
Time slot 20
    CPU 2: Put process 4 to run queue
    CPU 2: Dispatched process 6
    CPU 1: Put process 2 to run queue
    CPU 1: Dispatched process 4
    CPU 0: Put process 3 to run queue
    CPU 0: Dispatched process 2
Time slot 21
    CPU 3: Put process 5 to run queue
    CPU 3: Dispatched process 3
    CPU 2: Put process 6 to run queue
    CPU 2: Dispatched process 5
Time slot 22
    CPU 1: Put process 4 to run queue
    CPU 1: Dispatched process 6
Time slot 23
    CPU 0: Put process 2 to run queue
    CPU 0: Dispatched process 4
    CPU 3: Put process 3 to run queue
Time slot 24
    CPU 2: Put process 5 to run queue
    CPU 2: Dispatched process 3
    CPU 3: Dispatched process 2
    CPU 2: Processed 3 has finished
    CPU 2: Dispatched process 5
    CPU 1: Put process 6 to run queue
    CPU 1: Dispatched process 6
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 4
    CPU 3: Processed 2 has finished
```

CPU 3 stopped

A.3 Process Completion and CPU Stop Events

Listing 12 shows the final completion events. These lines are the source for the observed completion table in Section 8.

Listing 12: Completion and CPU-stop events from `os_2_mlq_paging.output`.

```
Time slot 25
    CPU 1: Processed 6 has finished
    CPU 1 stopped
Time slot 26
    CPU 2: Put process 5 to run queue
    CPU 2: Dispatched process 5
    CPU 0: Put process 4 to run queue
    CPU 0: Dispatched process 4
libfree:219
[LIBFREE] PID 4: free [1]
Time slot 27
Time slot 28
    CPU 2: Put process 5 to run queue
    CPU 0: Processed 4 has finished
    CPU 0 stopped
Time slot 29
    CPU 2: Dispatched process 5
Time slot 30
Time slot 31
    CPU 2: Processed 5 has finished
    CPU 2 stopped
```

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne. *Operating System Concepts*. 10th ed. Wiley, 2018.
- [2] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology. *CO2018 Operating Systems Assignment: Simple Operating System*. 2026.
- [3] Project source code. *36OS-main*. Available at: <https://github.com/baohuynhgia2006-jpg/36OS>.